DEMO USING ASK MODE

Selezionare database Bergamo_2026

PROMPT 1 - INITIAL QUERY STORE PERFORMANCE OVERVIEW

I want to start a SQL performance troubleshooting investigation using Query Store.

Generate a single read-only T-SQL query that I can execute in the current database to identify the main performance contributors.

The query must use Query Store and return, for each query:

* Query ID
* Query text
* Object or stored procedure name, when available
* Number of execution plans
* Number of executions
* Total CPU time
* Average CPU time per execution
* Total duration
* Average duration per execution
* Total logical reads
* Average logical reads per execution

Aggregate Query Store runtime statistics correctly across runtime intervals and execution plans.

Use weighted calculations based on count_executions.
Do not calculate averages by simply averaging Query Store average values.

Return CPU and duration values in milliseconds.

Rank the results by total CPU consumption, but include all the metrics above so that we can determine later which resource is having the highest impact.

Do not perform root-cause analysis yet.

Do not include execution plan analysis, parameter sensitivity, statistics analysis, index analysis, or remediation recommendations.

Only generate the read-only diagnostic T-SQL query that I should execute.

OUTPUT:

```
-- Created by GitHub Copilot in SSMS - review carefully before executing
WITH QueryStoreAggregates AS
(
    SELECT
        qsq.query_id,
        qst.query_sql_text,
        CASE
            WHEN qsq.object_id IS NULL OR qsq.object_id = 0 THEN NULL
            ELSE QUOTENAME(OBJECT_SCHEMA_NAME(qsq.object_id)) + N'.' + QUOTENAME(OBJECT_NAME(qsq.object_id))
        END AS object_name,
        COUNT(DISTINCT qsp.plan_id) AS execution_plan_count,
        SUM(CONVERT(bigint, rs.count_executions)) AS execution_count,
        SUM(CONVERT(decimal(38,6), rs.avg_cpu_time) * CONVERT(decimal(38,6), rs.count_executions)) AS total_cpu_us,
        SUM(CONVERT(decimal(38,6), rs.avg_duration) * CONVERT(decimal(38,6), rs.count_executions)) AS total_duration_us,
        SUM(CONVERT(decimal(38,6), rs.avg_logical_io_reads) * CONVERT(decimal(38,6), rs.count_executions)) AS total_logical_reads
    FROM [sys].[query_store_query] AS qsq
    INNER JOIN [sys].[query_store_query_text] AS qst
        ON qst.query_text_id = qsq.query_text_id
    INNER JOIN [sys].[query_store_plan] AS qsp
        ON qsp.query_id = qsq.query_id
    INNER JOIN [sys].[query_store_runtime_stats] AS rs
        ON rs.plan_id = qsp.plan_id
    WHERE rs.execution_type = 0
    GROUP BY
        qsq.query_id,
        qst.query_sql_text,
        qsq.object_id
)
SELECT
    ROW_NUMBER() OVER (ORDER BY qsa.total_cpu_us DESC, qsa.query_id) AS cpu_rank,
    qsa.query_id AS query_id,
    qsa.query_sql_text AS query_text,
    qsa.object_name AS object_or_procedure_name,
    qsa.execution_plan_count AS number_of_execution_plans,
    qsa.execution_count AS number_of_executions,
    CAST(qsa.total_cpu_us / 1000.0 AS decimal(19,2)) AS total_cpu_ms,
    CAST((qsa.total_cpu_us / NULLIF(qsa.execution_count, 0)) / 1000.0 AS decimal(19,2)) AS avg_cpu_ms_per_execution,
    CAST(qsa.total_duration_us / 1000.0 AS decimal(19,2)) AS total_duration_ms,
    CAST((qsa.total_duration_us / NULLIF(qsa.execution_count, 0)) / 1000.0 AS decimal(19,2)) AS avg_duration_ms_per_execution,
    CAST(qsa.total_logical_reads AS decimal(19,2)) AS total_logical_reads,
    CAST(qsa.total_logical_reads / NULLIF(qsa.execution_count, 0) AS decimal(19,2)) AS avg_logical_reads_per_execution
FROM QueryStoreAggregates AS qsa
ORDER BY
    qsa.total_cpu_us DESC,
    qsa.query_id;
```

PROMPT 2 - IDENTIFY THE DOMINANT PERFORMANCE METRIC

Using the results currently returned in the query grid, determine which performance metric is showing the greatest overall impact.

Consider:

* CPU time
* Duration
* Logical reads

```
* Execution volume

At this stage, do not identify or name any specific query, Query ID, stored procedure, or object.

Only tell me:

* Which metric appears to be dominant
* The evidence supporting that conclusion
* Which metric should be investigated first and why

Do not investigate individual queries yet.

Do not perform execution plan analysis, parameter sensitivity analysis, statistics analysis, index analysis, or remediation.
```

OUTPUT:

Based on the current Query Store result set, the **dominant metric is CPU time**.

Area	Evidence from current grid	Interpretation
CPU (total)	The top contributor accounts for **~98% of total CPU** in the returned set (orders of magnitude above the rest).	CPU is the strongest overall pressure signal.
Duration (total)	Also highly concentrated in the same workload (**~90%+** of total duration), but less extreme than CPU concentration.	Duration is significant, but appears secondary to CPU concentration.
Logical reads (total)	Elevated and concentrated, but distribution is less skewed than CPU.	IO contributes, but does not dominate as strongly as CPU.
Execution volume	One workload has **very high execution count** versus all others (roughly **~99%** of executions).	High frequency is likely amplifying cumulative CPU impact.

What to investigate first

Investigate **CPU consumption first**.

Why

The data shows the clearest and largest concentration in total CPU, and execution volume suggests this is a **cumulative CPU pressure pattern** (many executions), making CPU the highest-impact starting point for the next step.

PROMPT 3 - IDENTIFY THE MAIN CONTRIBUTING QUERY , Now focus on the performance metric identified in PROMPT 2.

```
Using the current Query Store results, identify which query contributes the most to that metric.

Show:

* Query ID
* Query text
* Object or stored procedure name
* Total consumption of the identified metric
* Average consumption per execution
* Number of executions
* Total duration
* Number of execution plans

Only identify the main contributor.

Do not investigate why the query is expensive yet.

Do not analyze execution plans, parameters, statistics, indexes, data distribution, or possible root causes.

Do not provide remediation recommendations.
```

OUTPUT:

Main contributor for the dominant metric (**CPU time**) from the current Query Store results:

Query ID	Query text	Object / procedure name	Total CPU (ms)	Avg CPU per execution (ms)	Executions	Total duration (ms)	Execution plans
1	**(@N2 int,@N1 int)SELECT count(Name),name, 'easyfilter' AS easyfilter FROM Notes where ID BETWEEN @N2 AND @N1 group by Name**	**[dbo].[GiveNotes]**	1,382,706.82	120.06	11,517	309,140.78	4

PROMPT 4 - AUTOMATICALLY IDENTIFY THE TARGET QUERY AND ALL OF ITS EXECUTION PLANS

```
Generate a single read-only T-SQL batch that I can execute in the current database to analyze the query identified as the main contributor in PROMPT 3.

IMPORTANT:

Do not require me to provide a Query ID.
Do not require me to provide any Plan IDs.
Do not hard-code Query IDs or Plan IDs.

The T-SQL must automatically identify the target query from Query Store using the performance metric identified in PROMPT 2 and the main contributor identified in PROMPT 3.

Once the target query has been identified, automatically retrieve ALL execution plans associated with that query.
```

The query must include details per-interval min/max runtime stats for the same plan over time, to verify if any single Plan ID degrade significantly over the time

OUTPUT:

Three T-SQL statements,

```
-- Created by GitHub Copilot in SSMS - review carefully before executing
;WITH QueryCpuAgg AS
(
    SELECT
        qsq.query_id,
        qsq.query_text_id,
        qsq.object_id,
        SUM(CONVERT(bigint, rs.count_executions)) AS total_executions,
        SUM(CONVERT(decimal(38,6), rs.avg_cpu_time) * CONVERT(decimal(38,6), rs.count_executions)) AS total_cpu_us
    FROM sys.query_store_query AS qsq
    INNER JOIN sys.query_store_plan AS qsp
        ON qsp.query_id = qsq.query_id
    INNER JOIN sys.query_store_runtime_stats AS rs
        ON rs.plan_id = qsp.plan_id
    WHERE rs.execution_type = 0
    GROUP BY
        qsq.query_id,
        qsq.query_text_id,
        qsq.object_id
),
TargetQuery AS
(
    SELECT TOP (1)
        qa.query_id,
        qa.query_text_id,
        qa.object_id
    FROM QueryCpuAgg AS qa
    ORDER BY
        qa.total_cpu_us DESC,
        qa.query_id
),
PlanIntervalStats AS
(
    SELECT
        tq.query_id,
        qsp.plan_id,
        qsp.plan_group_id,
        qsp.query_plan_hash,
        qsp.engine_version,
        qsp.compatibility_level,
        qsp.is_forced_plan,
        qsp.is_parallel_plan,
        qsp.is_trivial_plan,
        rsi.start_time AS interval_start_time_utc,
        rsi.end_time AS interval_end_time_utc,
        SUM(CONVERT(bigint, rs.count_executions)) AS interval_execution_count,
        SUM(CONVERT(decimal(38,6), rs.avg_cpu_time) * CONVERT(decimal(38,6), rs.count_executions)) AS interval_total_cpu_us,
        SUM(CONVERT(decimal(38,6), rs.avg_duration) * CONVERT(decimal(38,6), rs.count_executions)) AS interval_total_duration_us,
        SUM(CONVERT(decimal(38,6), rs.avg_logical_io_reads) * CONVERT(decimal(38,6), rs.count_executions)) AS interval_total_logical_reads,
        MIN(CONVERT(decimal(38,6), rs.min_cpu_time)) AS interval_min_cpu_us,
        MAX(CONVERT(decimal(38,6), rs.max_cpu_time)) AS interval_max_cpu_us,
        MIN(CONVERT(decimal(38,6), rs.min_duration)) AS interval_min_duration_us,
        MAX(CONVERT(decimal(38,6), rs.max_duration)) AS interval_max_duration_us,
        MIN(CONVERT(decimal(38,6), rs.min_logical_io_reads)) AS interval_min_logical_reads,
        MAX(CONVERT(decimal(38,6), rs.max_logical_io_reads)) AS interval_max_logical_reads
    FROM TargetQuery AS tq
    INNER JOIN sys.query_store_plan AS qsp
        ON qsp.query_id = tq.query_id
    INNER JOIN sys.query_store_runtime_stats AS rs
        ON rs.plan_id = qsp.plan_id
    INNER JOIN sys.query_store_runtime_stats_interval AS rsi
        ON rsi.runtime_stats_interval_id = rs.runtime_stats_interval_id
    WHERE rs.execution_type = 0
    GROUP BY
        tq.query_id,
        qsp.plan_id,
        qsp.plan_group_id,
        qsp.query_plan_hash,
        qsp.engine_version,
        qsp.compatibility_level,
        qsp.is_forced_plan,
        qsp.is_parallel_plan,
        qsp.is_trivial_plan,
        rsi.start_time,
        rsi.end_time
)
SELECT
    pis.query_id,
    LEFT(qst.query_sql_text, 100) AS query_text_100_chars,
    CASE
        WHEN tq.object_id IS NULL OR tq.object_id = 0 THEN NULL
        ELSE QUOTENAME(OBJECT_SCHEMA_NAME(tq.object_id)) + N'.' + QUOTENAME(OBJECT_NAME(tq.object_id))
    END AS object_or_procedure_name,
    pis.plan_id,
    pis.plan_group_id,
    pis.query_plan_hash,
    pis.engine_version,
    pis.compatibility_level,
    pis.is_forced_plan,
    pis.is_parallel_plan,
    pis.is_trivial_plan,
    pis.interval_start_time_utc,
    pis.interval_end_time_utc,
    pis.interval_execution_count,
    CAST(pis.interval_total_cpu_us / 1000.0 AS decimal(19,2)) AS interval_total_cpu_ms,
    CAST((pis.interval_total_cpu_us / NULLIF(pis.interval_execution_count, 0)) / 1000.0 AS decimal(19,2)) AS interval_avg_cpu_ms_per_execution,
    CAST(pis.interval_min_cpu_us / 1000.0 AS decimal(19,2)) AS interval_min_cpu_ms,
    CAST(pis.interval_max_cpu_us / 1000.0 AS decimal(19,2)) AS interval_max_cpu_ms,
    CAST(pis.interval_total_duration_us / 1000.0 AS decimal(19,2)) AS interval_total_duration_ms,
    CAST((pis.interval_total_duration_us / NULLIF(pis.interval_execution_count, 0)) / 1000.0 AS decimal(19,2)) AS interval_avg_duration_ms_per_execution,
    CAST(pis.interval_min_duration_us / 1000.0 AS decimal(19,2)) AS interval_min_duration_ms,
    CAST(pis.interval_max_duration_us / 1000.0 AS decimal(19,2)) AS interval_max_duration_ms,
    CAST(pis.interval_total_logical_reads AS decimal(19,2)) AS interval_total_logical_reads,
    CAST(pis.interval_total_logical_reads / NULLIF(pis.interval_execution_count, 0) AS decimal(19,2)) AS interval_avg_logical_reads_per_execution,
    CAST(pis.interval_min_logical_reads AS decimal(19,2)) AS interval_min_logical_reads,
    CAST(pis.interval_max_logical_reads AS decimal(19,2)) AS interval_max_logical_reads,
    SUM(pis.interval_execution_count) OVER (PARTITION BY pis.plan_id) AS plan_total_executions,
    CAST(SUM(pis.interval_total_cpu_us) OVER (PARTITION BY pis.plan_id) / 1000.0 AS decimal(19,2)) AS plan_total_cpu_ms,
    CAST(
        (SUM(pis.interval_total_cpu_us) OVER (PARTITION BY pis.plan_id)
         / NULLIF(SUM(pis.interval_execution_count) OVER (PARTITION BY pis.plan_id), 0)) / 1000.0
        AS decimal(19,2)
    ) AS plan_avg_cpu_ms_per_execution,
    CAST(MIN(pis.interval_min_cpu_us) OVER (PARTITION BY pis.plan_id) / 1000.0 AS decimal(19,2)) AS plan_min_cpu_ms_seen,
```

```
    CAST(MAX(pis.interval_max_cpu_us) OVER (PARTITION BY pis.plan_id) / 1000.0 AS decimal(19,2)) AS plan_max_cpu_ms_seen,
    CAST(SUM(pis.interval_total_duration_us) OVER (PARTITION BY pis.plan_id) / 1000.0 AS decimal(19,2)) AS plan_total_duration_ms,
    CAST(
        (SUM(pis.interval_total_duration_us) OVER (PARTITION BY pis.plan_id)
         / NULLIF(SUM(pis.interval_execution_count) OVER (PARTITION BY pis.plan_id), 0)) / 1000.0
        AS decimal(19,2)
    ) AS plan_avg_duration_ms_per_execution,
    CAST(MIN(pis.interval_min_duration_us) OVER (PARTITION BY pis.plan_id) / 1000.0 AS decimal(19,2)) AS plan_min_duration_ms_seen,
    CAST(MAX(pis.interval_max_duration_us) OVER (PARTITION BY pis.plan_id) / 1000.0 AS decimal(19,2)) AS plan_max_duration_ms_seen,
    CAST(SUM(pis.interval_total_logical_reads) OVER (PARTITION BY pis.plan_id) AS decimal(19,2)) AS plan_total_logical_reads,
    CAST(
        SUM(pis.interval_total_logical_reads) OVER (PARTITION BY pis.plan_id)
        / NULLIF(SUM(pis.interval_execution_count) OVER (PARTITION BY pis.plan_id), 0)
        AS decimal(19,2)
    ) AS plan_avg_logical_reads_per_execution,
    CAST(MIN(pis.interval_min_logical_reads) OVER (PARTITION BY pis.plan_id) AS decimal(19,2)) AS plan_min_logical_reads_seen,
    CAST(MAX(pis.interval_max_logical_reads) OVER (PARTITION BY pis.plan_id) AS decimal(19,2)) AS plan_max_logical_reads_seen
FROM PlanIntervalStats AS pis
INNER JOIN TargetQuery AS tq
    ON tq.query_id = pis.query_id
INNER JOIN sys.query_store_query_text AS qst
    ON qst.query_text_id = tq.query_text_id
ORDER BY
    pis.plan_id,
    pis.interval_start_time_utc;
```

once executed, return two result sets.

RESULT SET 1 - TARGET QUERY

QueryID	QueryText	ObjectOrProcedureName	ExecutionPlanCount	ExecutionCount	…
1	(@N1 int,@N2 int)SELECT count(Name),name, 'easyfilter' AS easyfilter FROM Notes where ID<@N1 AND ID>@N2 group by Name	[dbo].[GiveNotes]	4	11250	…
6	INSERT INTO Notes (ID,Name) SELECT RAND()*(100000 - 1) + 1, 'Info:'+convert(varchar(200),RAND()*(100000 - 1) + 1) FROM Notes	NULL	4	19	…
16	(@samplePercent float)SELECT StatMan([SC0], [SB0000]) FROM (SELECT TOP 100 PERCENT [SC0], step_direction([SC0]) over (order by NULL) AS [SB0000] FROM (SELECT [NAME] AS [SC0] FROM [dbo].[Notes] TABLESAMPLE SYSTEM (@samplePercent PERCENT) WITH (READUNCOMMITTED)) AS _MS_UPDSTATS_TBL_HELPER ORDER BY [SC0], [SB0000]) AS _MS_UPDSTATS_TBL OPTION (MAXDOP 8, RECOMPILE)	NULL	1	2	…

RESULT SET 2 - TARGET QUERY

TargetQueryID	QueryTextFirst100Chars	ObjectOrProcedureName	TotalExecutionPlansForTargetQuery	PlanID	…
1	(@N1 int,@N2 int)SELECT count(Name),name, 'easyfilter' AS easyfilter FROM Notes where ID<@N1 AND ID>	[dbo].[GiveNotes]	4	23	…
1	(@N1 int,@N2 int)SELECT count(Name),name, 'easyfilter' AS easyfilter FROM Notes where ID<@N1 AND ID>	[dbo].[GiveNotes]	4	19	…
1	(@N1 int,@N2 int)SELECT count(Name),name, 'easyfilter' AS easyfilter FROM Notes where ID<@N1 AND ID>	[dbo].[GiveNotes]	4	14	…
1	(@N1 int,@N2 int)SELECT count(Name),name, 'easyfilter' AS easyfilter FROM Notes where ID<@N1 AND ID>	[dbo].[GiveNotes]	4	1	…

PROMPT 5 - ANALYZE ALL AUTOMATICALLY DISCOVERED PLANS

Using the Query Store result grids currently displayed from PROMPT 4, analyze all execution plans associated with the automatically identified target query.

Use the Query ID and Plan IDs directly from the current result grids.

Do not ask me to provide any Query ID or Plan ID.

Compare:

* Number of executions
* Average CPU time
* Average duration
* Average logical reads
* Minimum and maximum values
* Performance over the runtime intervals

Identify:

* Material differences between plans; output in a grid format, including speed comparison ratio between plans
* Which plans show the lowest runtime cost
* Which plans show the highest runtime cost
* Whether performance degradation started before or after a different plan appeared, including comment if this issue should be investigated
* Whether any single Plan ID shows significant degradation while that same Plan ID remained in use, including comment if this issue should be investigated

Show the numeric evidence supporting the conclusions.

Do not investigate root cause yet.

Do not analyze parameter sensitivity, statistics, indexes, or remediation.

OUTPUT:

Plan comparison (auto-identified target query, from current grid)

Plan ID	Executions	Avg CPU (ms/exec)	Avg Duration (ms/exec)	Avg Logical Reads (reads/exec)	Min CPU (ms)	Max CPU (ms)	Min Duration (ms)	Max Duration (ms)	Min Reads	Max Reads	CPU ratio vs best plan*
1	5,102	0.26	0.27	2.23	0.04	4.06	0.04	4.79	2	11	1.0x
6	3,914	9.98	10.22	66.09	0.62	148.61	0.62	626.04	11	1,073	38.4x
11	813	83.31	86.95	1,773.65	24.05	234.28	24.05	1,693.86	1,075	4,159	320.4x
14	1,688	755.07	116.75	11,436.20	114.45	4,121.84	18.61	9,970.07	4,224	34,738	2,904.1x

*CPU ratio uses plan_avg_cpu_ms_per_execution relative to Plan 1.

Performance over runtime intervals

Plan ID	First interval avg CPU (ms)	Last interval avg CPU (ms)	Change	Comment
1	0.17	0.35	2.1x	Mild increase; low absolute cost.
6	4.97	72.68	14.6x	Large increase; should be investigated.
11	69.70	100.94	1.45x	Moderate increase; monitor/investigate.
14	280.39	2,564.88	9.15x	Severe increase; high priority to investigate.

Requested findings

Finding	Result	Numeric evidence	Investigate?
Lowest runtime cost plan	**Plan 1**	Lowest avg CPU (0.26), duration (0.27), reads (2.23)	No immediate urgency
Highest runtime cost plan	**Plan 14**	Avg CPU 755.07, total CPU 1,274,552.27, reads 11,436.20	**Yes (high priority)**
Degradation before/after different plan appeared	**After new higher-cost plan appeared**	Plan 14 appears at 13:50–13:51 already high (280.39 avg CPU), then rises to 2,564.88	**Yes**
Same Plan ID degrading while still in use	**Yes (Plan 14 clearly, Plan 6 also)**	Plan 14: 280.39 → 474.71 → 1,134.73 → 2,295.76 → 2,564.88; Plan 6: 4.97 → 72.68	**Yes**

Summary: Material plan-to-plan performance differences are extreme, and both **plan change timing** and **within-plan degradation (especially Plan 14)** warrant focused investigation in the next step.

PROMPT 6 - IDENTIFY SAME-PLAN PERFORMANCE DEGRADATION

```
Using the Query Store result grids generated in PROMPT 4 and the plan comparison performed in PROMPT 5, determine whether the target query experienced a significant performance degradation while the SAME Plan ID was still being used.

Do not ask me to provide a Query ID.

Do not ask me to provide a Plan ID.

Use the Query ID and all Plan IDs already present in the current Query Store result grids.

Inspect the runtime interval history for every execution plan associated with the target query.

Automatically identify the Plan ID, if any, that shows the clearest transition from healthy performance to degraded performance while that same Plan ID remained in use.

For that plan, compare the healthy period with the degraded period using:

* Average CPU time per execution
* Average duration per execution
* Average logical reads per execution
* Number of executions
* Runtime interval start and end times

Show the actual numeric values from the Query Store results.

Answer:

* Which Plan ID shows the clearest same-plan degradation?
* During which runtime interval was the plan performing normally?
* During which runtime interval did the degradation become clearly visible?
* How much did average CPU time change?
* How much did average duration change?
* How much did average logical reads change?
* Which metric changed the most?
* Did the degradation begin before another execution plan appeared?
* Can the beginning of the degradation be explained by a plan change alone?

Separate the response into:

1. Confirmed evidence
2. Observations
3. What the evidence does NOT yet prove
4. What should be investigated next
```

```
Do not assume parameter sensitivity.

Do not assume statistics changes.

Do not assume data growth or data distribution changes.

Do not assume an index problem.

Do not provide remediation recommendations yet.
```

OUTPUT:

1) Confirmed evidence

Clearest same-plan degradation: Plan ID 14

Period	Interval (UTC)	Executions	Avg CPU ms/exec	Avg Duration ms/exec	Avg Logical Reads/exec
Healthy baseline	2026-10-27 13:50:00 to 13:51:00	414	280.39	43.95	4,224.00
Degraded (clearly visible)	2026-10-27 13:52:00 to 13:53:00	329	1,134.73	177.14	17,298.53
Degraded (severe)	2026-10-27 13:54:00 to 13:55:00	19	2,564.88	339.61	34,738.00

Change from healthy baseline to severe degraded:

Metric	Healthy	Degraded	Absolute Change	Multiplicative Change
Avg CPU ms/exec	280.39	2,564.88	+2,284.49	**9.15x**
Avg Duration ms/exec	43.95	339.61	+295.66	7.73x
Avg Logical Reads/exec	4,224.00	34,738.00	+30,514.00	8.22x

Metric with largest relative change: Average CPU time per execution (9.15x).

---

2) Observations

Question	Answer
Which Plan ID shows clearest same-plan degradation?	**Plan 14**
When was performance normal?	First Plan 14 interval: **13:50–13:51**
When did degradation become clearly visible?	By **13:52–13:53** (and worse in later intervals)
Did degradation begin before another plan appeared?	**No newer plan appears after Plan 14 in the shown data; degradation escalates while Plan 14 remains active.**
Can beginning of degradation be explained by plan change alone?	**Not fully.** There is an initial jump when Plan 14 appears, but substantial additional worsening occurs **within the same Plan ID**.

---

3) What the evidence does not yet prove

- It does **not** prove the root cause.
- It does **not** prove parameter sensitivity.
- It does **not** prove statistics changes.
- It does **not** prove data growth/distribution effects.
- It does **not** prove an index issue.

4) What should be investigated next

- Validate whether workload characteristics changed by interval (input range/row counts/concurrency) while Plan 14 stayed active.
- Correlate interval-level CPU/read spikes with runtime context (time window pressure, concurrent activity).
- Compare Plan 14 early vs late interval runtime profile signals (same plan, different runtime behavior) before moving into root-cause domains.

PROMPT 7 - INVESTIGATE WHY THE SAME PLAN IS DOING MORE WORK

```
Re-evaluate the evidence collected in PROMPTS 1 through 6 for the target query and the execution plan that showed the clearest same-plan degradation.

Do not ask me to provide a Query ID or Plan ID.

Use the query text and Query Store evidence already available in the conversation and current result grids.

The previous investigation showed that performance changed significantly while the same execution plan continued to be reused.

Analyze whether the change appears to represent more work being performed per execution.

Focus especially on the relationship between:

* CPU time per execution
* Duration per execution
* Logical reads per execution

Answer:

* Could increased waits alone explain a large increase in logical reads per execution?
* Does the evidence indicate that SQL Server is processing significantly more data or performing significantly more work per execution?
```

```
* Inspect the query text. Which predicates, parameters, joins, grouping operations, or other query characteristics could cause the amount of work performed by the SAME plan to vary significantly between executions?
* Which explanation is currently the strongest based on the evidence?
* What evidence is still missing before that explanation can be considered confirmed?
* What read-only database information should be collected next to validate or reject that explanation?

Separate the response into:

1. Confirmed evidence
2. Inference from the evidence
3. Strongest hypothesis
4. Missing evidence
5. Next validation

Do not provide remediation recommendations yet.

Do not assume parameter sniffing merely because parameters are present.

Do not assume statistics or indexes are responsible until additional evidence supports that conclusion.
```

OUTPUT:

1) Confirmed evidence

Item	Healthy interval (Plan 14)	Degraded interval (Plan 14)	Change
Avg CPU / execution	280.39 ms	2,564.88 ms	**9.15x**
Avg duration / execution	43.95 ms	339.61 ms	**7.73x**
Avg logical reads / execution	4,224.00	34,738.00	**8.22x**

Query text (first 100 chars previously captured):
SELECT count(Name), name ... FROM Notes WHERE ID BETWEEN @N2 AND @N1 GROUP BY Name

---

2) Inference from the evidence

- **Increased waits alone are not a sufficient explanation** for the observed pattern.
  - Waits can increase duration, but they do **not** by themselves cause an ~8.22x increase in logical reads per execution.
- The joint rise in **CPU + duration + logical reads per execution** strongly indicates **more work per execution** (more rows/pages processed, more aggregation work, or both).

3) Strongest hypothesis

The strongest current explanation is **workload-shape variation under the same plan**:

- The `ID BETWEEN @N2 AND @N1` predicate can produce very different range sizes/selectivity between executions.
- `GROUP BY Name` can have very different grouping cardinality depending on rows returned.
- Therefore, the same physical plan can be reused while processing much more data for some executions, driving higher reads and CPU.

This is stronger than a pure wait-based explanation because logical reads/exec increased sharply.

4) Missing evidence

To confirm (or reject) that explanation, key missing evidence is:

Missing evidence	Why it matters
Per-interval row volume (`avg_rowcount`, min/max rowcount)	Confirms whether later executions returned/processed more rows.
Per-interval wait profile for this query/plan (`sys.query_store_wait_stats`)	Quantifies whether waits were dominant vs secondary.
Parameter value patterns for `@N1`, `@N2` during healthy vs degraded windows	Validates whether wider/different ranges align with higher work.
Per-interval memory usage and DOP metrics (`avg_query_max_used_memory`, `avg_dop`)	Helps detect runtime condition shifts that could amplify cost.

5) Next validation (read-only)

Collect and correlate, for the same query and Plan 14, by runtime interval:

1. Query Store runtime stats already used **plus** `avg_rowcount`, `min_rowcount`, `max_rowcount`, `avg_dop`, `avg_query_max_used_memory`.
2. Query Store wait stats (`sys.query_store_wait_stats`) by `plan_id` and interval.
3. Application-side execution inputs (or lightweight tracing) for `@N2`/`@N1` ranges during those exact UTC intervals.

If rowcount/range size rises in step with logical reads and CPU, the “same plan, more work” explanation is strongly supported.

PROMPT 8 - GENERATE A READ-ONLY VALIDATION BATCH FOR THE STRONGEST HYPOTHESIS

```
Using the strongest hypothesis identified in PROMPT 7, generate one self-contained read-only T-SQL batch that I can execute in the current database to validate or reject that hypothesis.

I understand that you cannot execute ad-hoc T-SQL directly in this session.

Do not ask me to provide a Query ID.

Do not ask me to provide any Plan IDs.

Do not require me to manually edit the generated T-SQL.

Generate the T-SQL to avoid potential issues related to message “Cannot insert the value NULL into column 'SchemaName', table” while writing into temporary table
Generate the T-SQL to avoid potential issues related to message “Multiple ordered aggregate functions in the same scope have mutually incompatible orderings.”
```

Use the target query already identified during this investigation and automatically locate the corresponding Query Store information.

The diagnostic batch should collect the evidence necessary to understand why the amount of work performed by the query can vary significantly between executions.

Adapt the diagnostic query to the evidence and strongest hypothesis already identified.

Where relevant, collect:

* Target Query ID and query text
* All execution Plan IDs
* Referenced table or tables
* Current row counts
* Existing indexes relevant to the query predicates
* Index key columns
* Included columns
* Statistics associated with the relevant predicate columns
* Whether statistics are auto-created, user-created, or associated with an index
* Statistics last update time
* Statistics row count
* Rows sampled
* Modification counter, where available
* Data distribution or histogram information for the columns relevant to the query predicates, where available
* Relevant estimated cardinality information from the Query Store execution plans
* Compile-time parameter information from the execution plan when available

IMPORTANT:

Do not assume in advance that statistics, parameters, data distribution, or indexes are the root cause.

The purpose of the batch is to collect evidence that can confirm or reject the strongest hypothesis from PROMPT 7.

Return clearly separated result sets so that the evidence can be analyzed in the next prompt.

Do not make any changes.

Do not update statistics.

Do not create or drop indexes.

Do not force plans.

Do not modify Query Store.

Do not provide remediation recommendations.

Only generate the read-only diagnostic T-SQL batch.

OUTPUT:

```
-- Created by GitHub Copilot in SSMS - review carefully before executing
SET NOCOUNT ON;

IF OBJECT_ID('tempdb..#TargetQuery') IS NOT NULL DROP TABLE #TargetQuery;
IF OBJECT_ID('tempdb..#TargetPlans') IS NOT NULL DROP TABLE #TargetPlans;
IF OBJECT_ID('tempdb..#PlanIntervals') IS NOT NULL DROP TABLE #PlanIntervals;
IF OBJECT_ID('tempdb..#ReferencedTables') IS NOT NULL DROP TABLE #ReferencedTables;
IF OBJECT_ID('tempdb..#ColumnsOfInterest') IS NOT NULL DROP TABLE #ColumnsOfInterest;
IF OBJECT_ID('tempdb..#StatsOfInterest') IS NOT NULL DROP TABLE #StatsOfInterest;

-------------------------------------------------------------------------------
-- 1) Auto-identify target query (top total CPU)
-------------------------------------------------------------------------------
;WITH QueryCpuAgg AS
(
    SELECT
        qsq.query_id,
        qsq.query_text_id,
        qsq.object_id,
        SUM(CONVERT(bigint, rs.count_executions)) AS total_executions,
        SUM(CONVERT(decimal(38,6), COALESCE(rs.avg_cpu_time, 0.0)) * CONVERT(decimal(38,6), rs.count_executions)) AS total_cpu_us
    FROM sys.query_store_query AS qsq
    INNER JOIN sys.query_store_plan AS qsp
        ON qsp.query_id = qsq.query_id
    INNER JOIN sys.query_store_runtime_stats AS rs
        ON rs.plan_id = qsp.plan_id
    WHERE rs.execution_type = 0
      AND qsq.is_internal_query = 0
    GROUP BY
        qsq.query_id,
        qsq.query_text_id,
        qsq.object_id
)
SELECT TOP (1)
    qa.query_id,
    qa.query_text_id,
    qa.object_id,
    LEFT(qst.query_sql_text, 100) AS query_text_100_chars,
    qa.total_executions,
    CAST(qa.total_cpu_us / 1000.0 AS decimal(20,2)) AS total_cpu_ms
INTO #TargetQuery
FROM QueryCpuAgg AS qa
INNER JOIN sys.query_store_query_text AS qst
    ON qst.query_text_id = qa.query_text_id
ORDER BY
    qa.total_cpu_us DESC,
    qa.query_id ASC;

SELECT
    tq.query_id,
    tq.query_text_id,
    tq.object_id,
```

```
    CASE
        WHEN tq.object_id IS NULL OR tq.object_id = 0 THEN NULL
        ELSE QUOTENAME(OBJECT_SCHEMA_NAME(CONVERT(int, tq.object_id), DB_ID()))
           + N'.'
           + QUOTENAME(OBJECT_NAME(CONVERT(int, tq.object_id), DB_ID()))
    END AS object_or_procedure_name,
    tq.query_text_100_chars,
    tq.total_executions,
    tq.total_cpu_ms
FROM #TargetQuery AS tq;
-- Result Set 1

-------------------------------------------------------------------------------
-- 2) Plans and interval runtime metrics
-------------------------------------------------------------------------------
SELECT
    qsp.plan_id,
    qsp.plan_group_id,
    qsp.query_plan_hash,
    qsp.engine_version,
    qsp.compatibility_level,
    qsp.is_forced_plan,
    qsp.is_parallel_plan,
    qsp.is_trivial_plan,
    qsp.plan_forcing_type_desc,
    TRY_CONVERT(xml, qsp.query_plan) AS query_plan_xml
INTO #TargetPlans
FROM sys.query_store_plan AS qsp
INNER JOIN #TargetQuery AS tq
    ON tq.query_id = qsp.query_id;

;WITH PlanIntervals AS
(
    SELECT
        tp.plan_id,
        rsi.runtime_stats_interval_id,
        rsi.start_time AS interval_start_time_utc,
        rsi.end_time AS interval_end_time_utc,
        SUM(CONVERT(bigint, rs.count_executions)) AS interval_execution_count,
        SUM(CONVERT(decimal(38,6), COALESCE(rs.avg_cpu_time, 0.0)) * CONVERT(decimal(38,6), rs.count_executions)) / 1000.0 AS interval_total_cpu_ms,
        (SUM(CONVERT(decimal(38,6), COALESCE(rs.avg_cpu_time, 0.0)) * CONVERT(decimal(38,6), rs.count_executions))
         / NULLIF(SUM(CONVERT(decimal(38,6), rs.count_executions)), 0)) / 1000.0 AS interval_avg_cpu_ms,
        SUM(CONVERT(decimal(38,6), COALESCE(rs.avg_duration, 0.0)) * CONVERT(decimal(38,6), rs.count_executions)) / 1000.0 AS interval_total_duration_ms,
        (SUM(CONVERT(decimal(38,6), COALESCE(rs.avg_duration, 0.0)) * CONVERT(decimal(38,6), rs.count_executions))
         / NULLIF(SUM(CONVERT(decimal(38,6), rs.count_executions)), 0)) / 1000.0 AS interval_avg_duration_ms,
        SUM(CONVERT(decimal(38,6), COALESCE(rs.avg_logical_io_reads, 0.0)) * CONVERT(decimal(38,6), rs.count_executions)) AS interval_total_logical_reads,
        (SUM(CONVERT(decimal(38,6), COALESCE(rs.avg_logical_io_reads, 0.0)) * CONVERT(decimal(38,6), rs.count_executions))
         / NULLIF(SUM(CONVERT(decimal(38,6), rs.count_executions)), 0)) AS interval_avg_logical_reads,
        MIN(CONVERT(decimal(38,6), rs.min_cpu_time)) / 1000.0 AS interval_min_cpu_ms,
        MAX(CONVERT(decimal(38,6), rs.max_cpu_time)) / 1000.0 AS interval_max_cpu_ms,
        MIN(CONVERT(decimal(38,6), rs.min_duration)) / 1000.0 AS interval_min_duration_ms,
        MAX(CONVERT(decimal(38,6), rs.max_duration)) / 1000.0 AS interval_max_duration_ms,
        MIN(CONVERT(decimal(38,6), rs.min_logical_io_reads)) AS interval_min_logical_reads,
        MAX(CONVERT(decimal(38,6), rs.max_logical_io_reads)) AS interval_max_logical_reads,
        (SUM(CONVERT(decimal(38,6), COALESCE(rs.avg_rowcount, 0.0)) * CONVERT(decimal(38,6), rs.count_executions))
         / NULLIF(SUM(CONVERT(decimal(38,6), rs.count_executions)), 0)) AS interval_avg_rowcount,
        MIN(CONVERT(decimal(38,6), rs.min_rowcount)) AS interval_min_rowcount,
        MAX(CONVERT(decimal(38,6), rs.max_rowcount)) AS interval_max_rowcount,
        (SUM(CONVERT(decimal(38,6), COALESCE(rs.avg_dop, 0.0)) * CONVERT(decimal(38,6), rs.count_executions))
         / NULLIF(SUM(CONVERT(decimal(38,6), rs.count_executions)), 0)) AS interval_avg_dop,
        (SUM(CONVERT(decimal(38,6), COALESCE(rs.avg_query_max_used_memory, 0.0)) * CONVERT(decimal(38,6), rs.count_executions))
         / NULLIF(SUM(CONVERT(decimal(38,6), rs.count_executions)), 0)) AS interval_avg_query_max_used_memory_kb
    FROM #TargetPlans AS tp
    INNER JOIN sys.query_store_runtime_stats AS rs
        ON rs.plan_id = tp.plan_id
    INNER JOIN sys.query_store_runtime_stats_interval AS rsi
        ON rsi.runtime_stats_interval_id = rs.runtime_stats_interval_id
    WHERE rs.execution_type = 0
    GROUP BY
        tp.plan_id,
        rsi.runtime_stats_interval_id,
        rsi.start_time,
        rsi.end_time
)
SELECT
    pi.plan_id,
    pi.runtime_stats_interval_id,
    pi.interval_start_time_utc,
    pi.interval_end_time_utc,
    pi.interval_execution_count,
    pi.interval_total_cpu_ms,
    pi.interval_avg_cpu_ms,
    pi.interval_total_duration_ms,
    pi.interval_avg_duration_ms,
    pi.interval_total_logical_reads,
    pi.interval_avg_logical_reads,
    pi.interval_min_cpu_ms,
    pi.interval_max_cpu_ms,
    pi.interval_min_duration_ms,
    pi.interval_max_duration_ms,
    pi.interval_min_logical_reads,
    pi.interval_max_logical_reads,
    pi.interval_avg_rowcount,
    pi.interval_min_rowcount,
    pi.interval_max_rowcount,
    pi.interval_avg_dop,
    pi.interval_avg_query_max_used_memory_kb
INTO #PlanIntervals
FROM PlanIntervals AS pi;

SELECT
    tq.query_id,
    tp.plan_id,
    tp.plan_group_id,
    tp.query_plan_hash,
    tp.engine_version,
    tp.compatibility_level,
    tp.is_forced_plan,
    tp.is_parallel_plan,
    tp.is_trivial_plan,
    tp.plan_forcing_type_desc,
    SUM(pi.interval_execution_count) AS plan_total_executions,
    CAST(SUM(pi.interval_total_cpu_ms) AS decimal(20,2)) AS plan_total_cpu_ms,
    CAST(SUM(pi.interval_total_cpu_ms) / NULLIF(SUM(pi.interval_execution_count), 0) AS decimal(20,2)) AS plan_avg_cpu_ms_per_execution,
    CAST(SUM(pi.interval_total_duration_ms) / NULLIF(SUM(pi.interval_execution_count), 0) AS decimal(20,2)) AS plan_avg_duration_ms_per_execution,
    CAST(SUM(pi.interval_total_logical_reads) / NULLIF(SUM(pi.interval_execution_count), 0) AS decimal(20,2)) AS plan_avg_logical_reads_per_execution
FROM #TargetQuery AS tq
INNER JOIN #TargetPlans AS tp
    ON 1 = 1
INNER JOIN #PlanIntervals AS pi
    ON pi.plan_id = tp.plan_id
GROUP BY
    tq.query_id,
    tp.plan_id,
    tp.plan_group_id,
    tp.query_plan_hash,
    tp.engine_version,
    tp.compatibility_level,
```

```
    tp.is_forced_plan,
    tp.is_parallel_plan,
    tp.is_trivial_plan,
    tp.plan_forcing_type_desc
ORDER BY
    plan_avg_cpu_ms_per_execution DESC,
    tp.plan_id;
-- Result Set 2

SELECT
    tq.query_id,
    pi.plan_id,
    pi.runtime_stats_interval_id,
    pi.interval_start_time_utc,
    pi.interval_end_time_utc,
    pi.interval_execution_count,
    CAST(pi.interval_avg_cpu_ms AS decimal(20,2)) AS interval_avg_cpu_ms_per_execution,
    CAST(pi.interval_avg_duration_ms AS decimal(20,2)) AS interval_avg_duration_ms_per_execution,
    CAST(pi.interval_avg_logical_reads AS decimal(20,2)) AS interval_avg_logical_reads_per_execution,
    CAST(pi.interval_avg_rowcount AS decimal(20,2)) AS interval_avg_rowcount
FROM #TargetQuery AS tq
INNER JOIN #PlanIntervals AS pi
    ON 1 = 1
ORDER BY
    pi.plan_id,
    pi.interval_start_time_utc;
-- Result Set 3

-------------------------------------------------------------------------------
-- 3) Referenced tables and row counts
-------------------------------------------------------------------------------
CREATE TABLE #ReferencedTables
(
    source_type sysname NULL,
    database_name sysname NULL,
    schema_name sysname NULL,
    table_name sysname NULL,
    object_id int NULL
);
INSERT INTO #ReferencedTables (source_type, database_name, schema_name, table_name, object_id)
SELECT DISTINCT
    N'module_dependency',
    DB_NAME(),
    OBJECT_SCHEMA_NAME(d.referenced_id, DB_ID()),
    OBJECT_NAME(d.referenced_id, DB_ID()),
    CONVERT(int, d.referenced_id)
FROM #TargetQuery AS tq
INNER JOIN sys.sql_expression_dependencies AS d
    ON d.referencing_id = tq.object_id
WHERE tq.object_id IS NOT NULL
  AND tq.object_id > 0
  AND d.referenced_id IS NOT NULL
  AND OBJECTPROPERTY(d.referenced_id, 'IsUserTable') = 1;

WITH XMLNAMESPACES (DEFAULT 'http://schemas.microsoft.com/sqlserver/2004/07/showplan')
INSERT INTO #ReferencedTables (source_type, database_name, schema_name, table_name, object_id)
SELECT DISTINCT
    N'plan_xml',
    COALESCE(NULLIF(x.obj.value('@Database','sysname'), N''), DB_NAME()),
    NULLIF(x.obj.value('@Schema','sysname'), N''),
    NULLIF(x.obj.value('@Table','sysname'), N''),
    OBJECT_ID(
        QUOTENAME(COALESCE(NULLIF(x.obj.value('@Schema','sysname'), N''), N'dbo'))
        + N'.'
        + QUOTENAME(REPLACE(REPLACE(COALESCE(NULLIF(x.obj.value('@Table','sysname'), N''), N''), N'[', N''), N']', N''))
    )
FROM #TargetPlans AS tp
CROSS APPLY tp.query_plan_xml.nodes('//Object[@Table]') AS x(obj);

;WITH DistinctTables AS
(
    SELECT DISTINCT
        rt.database_name,
        rt.schema_name,
        rt.table_name,
        rt.object_id
    FROM #ReferencedTables AS rt
    WHERE rt.table_name IS NOT NULL
)
SELECT
    dt.database_name,
    dt.schema_name,
    dt.table_name,
    dt.object_id,
    CAST(SUM(COALESCE(ps.row_count, 0)) AS bigint) AS current_row_count
FROM DistinctTables AS dt
LEFT JOIN sys.dm_db_partition_stats AS ps
    ON ps.object_id = dt.object_id
   AND ps.index_id IN (0, 1)
GROUP BY
    dt.database_name,
    dt.schema_name,
    dt.table_name,
    dt.object_id
ORDER BY
    dt.schema_name,
    dt.table_name;
-- Result Set 4

-------------------------------------------------------------------------------
-- 4) Columns of interest from plan XML (no XQuery union)
-------------------------------------------------------------------------------
CREATE TABLE #ColumnsOfInterest
(
    schema_name sysname NULL,
    table_name sysname NULL,
    column_name sysname NULL,
    evidence_source nvarchar(50) NOT NULL
);

WITH XMLNAMESPACES (DEFAULT 'http://schemas.microsoft.com/sqlserver/2004/07/showplan')
INSERT INTO #ColumnsOfInterest (schema_name, table_name, column_name, evidence_source)
SELECT DISTINCT
    NULLIF(c.col.value('@Schema','sysname'), N''),
    NULLIF(c.col.value('@Table','sysname'), N''),
    NULLIF(c.col.value('@Column','sysname'), N''),
    N'SeekPredicate'
FROM #TargetPlans AS tp
CROSS APPLY tp.query_plan_xml.nodes('//SeekPredicates//ColumnReference') AS c(col)
WHERE c.col.exist('@Column') = 1;

WITH XMLNAMESPACES (DEFAULT 'http://schemas.microsoft.com/sqlserver/2004/07/showplan')
INSERT INTO #ColumnsOfInterest (schema_name, table_name, column_name, evidence_source)
SELECT DISTINCT
    NULLIF(c.col.value('@Schema','sysname'), N''),
    NULLIF(c.col.value('@Table','sysname'), N''),
    NULLIF(c.col.value('@Column','sysname'), N''),
```

```
    N'Predicate'
FROM #TargetPlans AS tp
CROSS APPLY tp.query_plan_xml.nodes('//Predicate//ColumnReference') AS c(col)
WHERE c.col.exist('@Column') = 1;

WITH XMLNAMESPACES (DEFAULT 'http://schemas.microsoft.com/sqlserver/2004/07/showplan')
INSERT INTO #ColumnsOfInterest (schema_name, table_name, column_name, evidence_source)
SELECT DISTINCT
    NULLIF(c.col.value('@Schema','sysname'), N''),
    NULLIF(c.col.value('@Table','sysname'), N''),
    NULLIF(c.col.value('@Column','sysname'), N''),
    N'GroupBy'
FROM #TargetPlans AS tp
CROSS APPLY tp.query_plan_xml.nodes('//GroupBy//ColumnReference') AS c(col)
WHERE c.col.exist('@Column') = 1;

SELECT DISTINCT
    coi.schema_name,
    coi.table_name,
    coi.column_name,
    coi.evidence_source
FROM #ColumnsOfInterest AS coi
WHERE coi.column_name IS NOT NULL
ORDER BY
    coi.schema_name,
    coi.table_name,
    coi.column_name,
    coi.evidence_source;
-- Result Set 5

-------------------------------------------------------------------------------
-- 5) Relevant indexes (key and included columns)
-------------------------------------------------------------------------------
SELECT
    sch.name AS schema_name,
    t.name AS table_name,
    i.name AS index_name,
    i.index_id,
    i.type_desc AS index_type_desc,
    i.is_primary_key,
    i.is_unique,
    ic.is_included_column,
    ic.key_ordinal,
    c.name AS column_name
FROM sys.tables AS t
INNER JOIN sys.schemas AS sch
    ON sch.schema_id = t.schema_id
INNER JOIN sys.indexes AS i
    ON i.object_id = t.object_id
INNER JOIN sys.index_columns AS ic
    ON ic.object_id = i.object_id
   AND ic.index_id = i.index_id
INNER JOIN sys.columns AS c
    ON c.object_id = ic.object_id
   AND c.column_id = ic.column_id
INNER JOIN #ColumnsOfInterest AS coi
    ON coi.column_name = c.name
   AND COALESCE(coi.table_name, t.name) = t.name
   AND COALESCE(coi.schema_name, sch.name) = sch.name
ORDER BY
    sch.name,
    t.name,
    i.index_id,
    ic.is_included_column,
    ic.key_ordinal,
    c.name;
-- Result Set 6

-------------------------------------------------------------------------------
-- 6) Statistics metadata and properties
-------------------------------------------------------------------------------
SELECT DISTINCT
    s.object_id,
    s.stats_id,
    sch.name AS schema_name,
    t.name AS table_name,
    s.name AS stats_name,
    c.name AS column_name,
    sc.stats_column_id,
    s.auto_created,
    s.user_created,
    s.no_recompute,
    s.has_filter,
    s.filter_definition,
    sp.last_updated,
    sp.rows AS stats_rows,
    sp.rows_sampled,
    sp.steps AS histogram_steps,
    sp.unfiltered_rows,
    sp.modification_counter
INTO #StatsOfInterest
FROM sys.tables AS t
INNER JOIN sys.schemas AS sch
    ON sch.schema_id = t.schema_id
INNER JOIN sys.stats AS s
    ON s.object_id = t.object_id
INNER JOIN sys.stats_columns AS sc
    ON sc.object_id = s.object_id
   AND sc.stats_id = s.stats_id
INNER JOIN sys.columns AS c
    ON c.object_id = sc.object_id
   AND c.column_id = sc.column_id
INNER JOIN #ColumnsOfInterest AS coi
    ON coi.column_name = c.name
   AND COALESCE(coi.table_name, t.name) = t.name
   AND COALESCE(coi.schema_name, sch.name) = sch.name
OUTER APPLY sys.dm_db_stats_properties(s.object_id, s.stats_id) AS sp;

SELECT
    soi.schema_name,
    soi.table_name,
    soi.stats_name,
    soi.column_name,
    soi.stats_column_id,
    soi.auto_created,
    soi.user_created,
    soi.no_recompute,
    soi.has_filter,
    soi.filter_definition,
    soi.last_updated,
    soi.stats_rows,
    soi.rows_sampled,
    soi.histogram_steps,
    soi.unfiltered_rows,
    soi.modification_counter
FROM #StatsOfInterest AS soi
ORDER BY
    soi.schema_name,
```

```
    soi.table_name,
    soi.stats_name,
    soi.stats_column_id;
-- Result Set 7

-------------------------------------------------------------------------------
-- 7) Histogram data for stats (FIXED: equal_rows)
-------------------------------------------------------------------------------
SELECT
    soi.schema_name,
    soi.table_name,
    soi.stats_name,
    soi.column_name,
    h.step_number,
    h.range_high_key,
    h.range_rows,
    h.equal_rows,
    h.distinct_range_rows,
    h.average_range_rows
FROM #StatsOfInterest AS soi
CROSS APPLY sys.dm_db_stats_histogram(soi.object_id, soi.stats_id) AS h
ORDER BY
    soi.schema_name,
    soi.table_name,
    soi.stats_name,
    soi.column_name,
    h.step_number;
-- Result Set 8

-------------------------------------------------------------------------------
-- 8) Cardinality and parameter evidence from Query Store plans
-------------------------------------------------------------------------------
WITH XMLNAMESPACES (DEFAULT 'http://schemas.microsoft.com/sqlserver/2004/07/showplan')
SELECT
    tq.query_id,
    tp.plan_id,
    stmt.value('@StatementId','int') AS statement_id,
    LEFT(stmt.value('@StatementText','nvarchar(max)'), 400) AS statement_text_400_chars,
    stmt.value('@CardinalityEstimationModelVersion','int') AS ce_model_version,
    TRY_CONVERT(decimal(20,4), stmt.value('@StatementSubTreeCost','nvarchar(50)')) AS statement_subtree_cost,
    TRY_CONVERT(decimal(20,4), stmt.value('@StatementEstRows','nvarchar(50)')) AS statement_est_rows
FROM #TargetQuery AS tq
INNER JOIN #TargetPlans AS tp
    ON 1 = 1
CROSS APPLY tp.query_plan_xml.nodes('//StmtSimple') AS s(stmt)
ORDER BY
    tp.plan_id,
    statement_id;
-- Result Set 9

WITH XMLNAMESPACES (DEFAULT 'http://schemas.microsoft.com/sqlserver/2004/07/showplan')
SELECT
    tq.query_id,
    tp.plan_id,
    p.value('@Column','sysname') AS parameter_name,
    p.value('@ParameterCompiledValue','nvarchar(4000)') AS parameter_compiled_value,
    p.value('@ParameterRuntimeValue','nvarchar(4000)') AS parameter_runtime_value
FROM #TargetQuery AS tq
INNER JOIN #TargetPlans AS tp
    ON 1 = 1
CROSS APPLY tp.query_plan_xml.nodes('//ParameterList/ColumnReference') AS pr(p)
ORDER BY
    tp.plan_id,
    parameter_name;
-- Result Set 10

-------------------------------------------------------------------------------
-- 9) Query Store wait profile by plan and interval
-------------------------------------------------------------------------------
SELECT
    tq.query_id,
    qsw.plan_id,
    qsw.runtime_stats_interval_id,
    rsi.start_time AS interval_start_time_utc,
    rsi.end_time AS interval_end_time_utc,
    qsw.wait_category_desc,
    qsw.execution_type,
    CASE qsw.execution_type
        WHEN 0 THEN N'Regular'
        WHEN 3 THEN N'Client aborted'
        WHEN 4 THEN N'Exception'
        WHEN 5 THEN N'Canceled'
        ELSE N'Other'
    END AS execution_type_desc,
    qsw.total_query_wait_time_ms,
    qsw.avg_query_wait_time_ms,
    qsw.min_query_wait_time_ms,
    qsw.max_query_wait_time_ms
FROM #TargetQuery AS tq
INNER JOIN #TargetPlans AS tp
    ON 1 = 1
INNER JOIN sys.query_store_wait_stats AS qsw
    ON qsw.plan_id = tp.plan_id
INNER JOIN sys.query_store_runtime_stats_interval AS rsi
    ON rsi.runtime_stats_interval_id = qsw.runtime_stats_interval_id
ORDER BY
    qsw.plan_id,
    qsw.runtime_stats_interval_id,
    qsw.wait_category_desc;
-- Result Set 11
```

PROMPT 9 - DETERMINE THE ROOT CAUSE FROM THE VALIDATION EVIDENCE

Using the result grids generated by PROMPT 8 together with the Query Store evidence collected in PROMPTS 1 through 7, determine the most likely explanation for the performance degradation.

Do not ask me to provide a Query ID or Plan ID.

Use the identifiers and evidence already present in the current results.

Evaluate:

* Query predicate behavior
* Estimated amount of data processed
* Data distribution
* Predicate selectivity

```
* Statistics coverage and freshness
* Existing indexes
* Index key order
* Included columns
* Logical reads
* CPU time
* Duration
* Execution plan behavior over time

Determine whether the evidence supports a scenario where different executions cause the same execution plan to process
substantially different numbers of rows.

Clearly distinguish between:

1. Confirmed evidence
2. Supported diagnosis
3. Remaining uncertainty

Then identify which physical access paths are currently available to SQL Server for the query.

Do not provide the final remediation yet.

Do not simply say "review the indexes."

Explain specifically whether the current indexes efficiently support the query predicates and required output columns.

If an index appears to be missing or insufficient, describe what characteristic an appropriate index would need, but do not
provide the CREATE INDEX statement yet.
```

OUTPUT:

1. Confirmed evidence

Evidence area	What is confirmed
Target query	query_id = 1, [dbo].[GiveNotes], text starts with SELECT count(Name), name ... FROM Notes WHERE ID BETWEEN @N2 AND @N1 GROUP BY Name
Dominant high-cost plan	plan_id = 14 with plan_avg_cpu_ms_per_execution = 755.07, plan_avg_logical_reads_per_execution = 11436.20
Same-plan degradation	For plan_id = 14, interval avg CPU rose 280.39 -> 2564.88 ms (~9.15x), duration 43.95 -> 339.61 ms (~7.73x), logical reads 4224 -> 34738 (~8.22x)
Work-per-exec signal	interval_avg_rowcount also increases for plan 14 (32.96 -> 38.32), while reads and CPU rise much faster
Plan behavior over time	New plans appear over time (1 -> 6 -> 11 -> 14), but major worsening continues **within the same plan_id 14**
Wait behavior	Parallelism and lock waits increase in degraded intervals, but this is concurrent with large logical-read growth
Table/index metadata	dbo.Notes has clustered PK on id2 (PK_Notes). No evidence of an index keyed on predicate/grouping columns (ID, NAME) from returned diagnostics

---

2. Supported diagnosis

The strongest diagnosis is:

Different executions are causing the same plan to process materially different underlying row/page volumes, and the current access path is inefficient for the predicate/grouping pattern.

Why this is supported:

- Predicate is a variable range: ID BETWEEN @N2 AND @N1.
- Grouping is on NAME; output cardinality can stay moderate while input scanned rows vary heavily.
- Logical reads/exec increase ~8.22x with CPU/exec ~9.15x on the **same plan_id**, which is consistent with "more data/work processed," not just waiting.
- Existing known index (PK_Notes on id2) does not align with predicate ID and grouping/output needs around NAME.

3. Remaining uncertainty

Area	Uncertainty
Parameter/runtime shape	Runtime parameter values per interval are not captured (compiled values exist, runtime values are NULL).
Statistics evidence	Stats/index result sets for columns of interest came back empty in this run, so freshness/coverage is not yet fully validated from those specific outputs.
Access-operator proof	Full operator-level plan details were not returned in the result sets, so operator names are inferred from cost/read patterns and available metadata.
Wait dominance	Waits increased (notably parallelism/lock), but evidence does not prove waits are primary cause vs secondary effect of larger work.

Physical access paths currently available to SQL Server (for this query)

Aspect	Current state	Assessment
Known index on dbo.Notes	Clustered PK on id2	Not aligned to WHERE ID BETWEEN...
Predicate support (ID)	No confirmed index keyed by ID	Likely forces broad scan/residual filtering behavior
Grouping/output support (GROUP BY NAME, count(NAME), NAME)	No confirmed index keyed to help both filter and grouping	Requires heavier read/CPU work as qualifying range grows
Parallel plan usage	plan_id 14 is parallel	Helps throughput but also shows high parallelism waits under heavy work

If an index is insufficient/missing, the needed characteristic is:
an access path whose **leading key matches the range predicate on ID**, with key/include design that minimizes extra reads for grouping/output on NAME (so SQL Server can process fewer pages per execution).

PROMPT 10 - PROVIDE CONCRETE CORRECTIVE ACTIONS

```
Based on the complete evidence collected in PROMPTS 1 through 9, provide concrete corrective actions for the target query.

Do not ask me for a Query ID or Plan ID.

Use the query, plans, tables, statistics, and indexes already identified during the investigation.

Prioritize actions according to the evidence collected.

For indexing, do not provide generic advice such as "review indexes" or "consider adding an index."

Inspect:

* The filtering predicates
* Predicate column order
* Columns required by the SELECT or aggregation
* Existing clustered and nonclustered indexes
* Existing statistics
* Expected reduction in logical reads
* Expected impact on CPU and duration

If an additional or modified index is justified by the evidence:

* Provide the exact CREATE INDEX statement
* Explain why each key column was selected
* Explain why any INCLUDE columns were selected
* Explain how the index would improve the access path
* Explain why it could reduce logical reads
* Explain the expected effect on CPU and duration
* Explain potential storage and write overhead

Also evaluate statistics.

If an existing auto-created statistic may become redundant because the proposed index creates equivalent statistics on the same leading key column, identify it as a candidate for further evaluation.

Do not recommend dropping a statistic merely because an index was created.

Distinguish index statistics from independent column statistics.

Remember that INCLUDE columns do not receive histogram statistics simply because they are included in an index.

Do not apply any changes.

Separate the response into:

1. Evidence
2. Diagnosis
3. Recommended corrective actions
4. Risks and trade-offs
5. Changes that should be tested
```

OUTPUT:

1. Evidence

Area	Evidence from your results
Target query	`[dbo].[GiveNotes]: ... FROM Notes WHERE ID BETWEEN @N2 AND @N1 GROUP BY Name`
Dominant degraded plan	`plan_id = 14`
Same-plan degradation	Avg CPU/exec: `280.39 -> 2564.88 ms` (9.15x), Avg reads/exec: `4224 -> 34738` (8.22x), Avg duration/exec: `43.95 -> 339.61 ms` (7.73x)
Work expansion signal	`interval_avg_rowcount` rises (`32.96 -> 38.32`), while reads/CPU rise much faster
Current physical design	`dbo.Notes` has clustered PK on `id2`; no confirmed supporting index on predicate column `ID` for this query pattern

---

2. Diagnosis

Most likely root cause is **inefficient access path for variable `ID` range filtering**, causing the same plan to process widely different amounts of data per execution.
That drives large increases in logical reads, then CPU, then duration.
The current clustered key (`id2`) does not align with the predicate `ID BETWEEN ...`, so SQL Server likely reads far more pages than needed when the selected range is larger/less selective.

---

3. Recommended corrective actions (prioritized)

Action A (highest priority): Add a nonclustered index aligned to predicate + output needs

```
-- Created by GitHub Copilot in SSMS - review carefully before executing
CREATE NONCLUSTERED INDEX IX_Notes_ID_INCLUDE_NAME
ON dbo.Notes (ID)
INCLUDE (NAME);
```

Why this index

- **Key column `ID`**: matches the filtering predicate (`BETWEEN @N2 AND @N1`) so optimizer can seek/range-scan instead of broad scans.
- **Include `NAME`**: query needs `NAME` for `GROUP BY` and output; including it avoids key lookups to clustered index (`id2`).

How it improves access path

- Enables efficient range access on `ID`.
- Keeps required columns at index leaf level (`ID`, `NAME`) for covered execution.
- Reduces base-table page reads substantially for many ranges.

Expected effect

- **Logical reads**: significant reduction (especially in high-read intervals).
- **CPU**: reduced from lower row/page processing.
- **Duration**: reduced from lower work and less contention side-effects.

Action B: Improve statistics coverage for estimation of grouped range workloads

If not already present, add multi-column stats to improve estimates when filtering by `ID` and grouping by `NAME`:

```
-- Created by GitHub Copilot in SSMS - review carefully before executing
CREATE STATISTICS ST_Notes_ID_NAME
ON dbo.Notes (ID, NAME);
```

Why

- Index stats on `IX_Notes_ID_INCLUDE_NAME` provide histogram on leading `ID` only.
- `NAME` in `INCLUDE` does **not** get histogram statistics.
- Multi-column stats can improve cardinality estimates for range + grouping interaction.

Action C: Evaluate (not drop) potentially overlapping auto stats after index creation

- After creating the new index, evaluate whether any auto-created stat on leading column `ID` becomes redundant.
- This is **candidate review only**; no automatic drop recommendation.

4. Risks and trade-offs

Area	Risk / Cost
Storage	Additional nonclustered index size on `dbo.Notes` (~8.3M rows)
DML overhead	Inserts/updates/deletes on `dbo.Notes` will be slower due to index maintenance
Maintenance	Extra index/statistics maintenance workload
Plan shifts	Query optimizer may change plan shape; validate regressions on other workloads

5. Changes that should be tested

1. **Baseline vs after-index** for the same query and comparable parameter ranges:
   - Avg logical reads/exec
   - Avg CPU ms/exec
   - Avg duration ms/exec
2. Re-check Query Store per plan/interval to confirm:
   - Lower `avg_logical_io_reads`
   - Lower `avg_cpu_time`
   - Reduced variance across intervals
3. Validate write impact on `dbo.Notes` insert/update workload.
4. Re-check stats metadata after changes and evaluate overlapping stats candidates (no immediate drops).

PROMPT 11 - DEFINE THE QUERY STORE VALIDATION PLAN

```
Based on the investigation and corrective action proposed in PROMPT 10, define how Query Store should be used to validate whether the change actually improves performance.

Do not assume that the change is successful.

Do not rely only on execution duration.

Compare the before-and-after behavior of the target query using:

* Number of executions
* Average CPU time per execution
* Total CPU time
* Average duration per execution
* Total duration
* Average logical reads per execution
* Total logical reads
* Execution Plan IDs
* Runtime intervals
* Performance variability

Explain what would constitute convincing evidence that the corrective action improved the query.

Also identify evidence that would indicate:

* No meaningful improvement
* A regression
* A new execution plan with worse behavior
* Reduced average cost but increased performance variability

Do not ask me to provide Query IDs or Plan IDs manually.

Use the target query already identified during this investigation.

Do not apply any changes.
```

OUTPUT:

Query Store validation plan for the proposed change

Use Query Store to compare **before vs after** for the same target query (`query_id = 1`, `[dbo].[GiveNotes]`) using equal time windows and weighted metrics.

1) Validation setup

Step	Requirement
Baseline window	Use a pre-change window with representative workload (for example, 24h).
Post-change window	Use the same duration and similar business period (same hour/day pattern).
Warm-up exclusion	Exclude first 15–30 minutes after deployment.
Scope	`execution_type = 0` only (regular executions).
Aggregation rule	Weighted by `count_executions` across plans and intervals.

2) Metrics to compare (before vs after)

Metric	Why it matters	Improvement signal
Number of executions	Confirms workload comparability	Similar volume, or normalize totals by exec count
Avg CPU per execution	Primary cost driver identified earlier	Meaningful reduction
Total CPU	Overall server impact	Reduction with comparable execution volume
Avg duration per execution	User-facing latency	Reduction
Total duration	Aggregate time consumed	Reduction
Avg logical reads per execution	Work done per call	Reduction (key evidence)
Total logical reads	I/O footprint	Reduction with comparable workload
Plan IDs used	Detect plan shape changes	Stable or better-cost plans replacing worse ones
Runtime intervals	Detect time-local regressions	Fewer high-cost intervals
Variability	Stability check	Lower spread (less volatility)

3) Variability checks to include

Check	Calculation idea	Interpretation
CPU variability	`max(interval_avg_cpu_ms) / min(interval_avg_cpu_ms)`	Lower ratio = more stable
Duration variability	`max(interval_avg_duration_ms) / min(interval_avg_duration_ms)`	Lower ratio = more stable
Read variability	`max(interval_avg_logical_reads) / min(interval_avg_logical_reads)`	Lower ratio = more stable
Interval outliers	Count intervals above baseline p95-equivalent band	Fewer outliers = better

4) Decision criteria

Outcome	Evidence pattern
Convincing improvement	Avg CPU/exec and avg reads/exec both materially lower; total CPU lower at comparable execution count; duration lower; variability not worse; no new high-cost plan dominating
No meaningful improvement	Changes are marginal/inconsistent across intervals; CPU/reads roughly unchanged after normalizing for volume
Regression	Avg CPU/exec, reads/exec, or duration/exec increase; more high-cost intervals; worse totals with similar volume
New worse plan	New `plan_id` appears and has higher avg CPU/reads/duration than prior dominant plan(s), and captures substantial executions
Lower average but higher instability	Means improve, but interval variability rises (larger max/min ratios, more spikes)

5) Specific pass/fail focus for this case

Given the investigation, treat these as primary validation signals:

1. **Avg logical reads/exec decreases** (must improve, not just duration).
2. **Avg CPU/exec decreases** in the same direction as reads.
3. **Plan behavior** does not shift to a new high-cost `plan_id`.
4. **Interval stability** improves or at least does not degrade.

If needed, I can provide a single read-only Query Store comparison query that returns all before/after metrics and variability in one grid.

PROMPT 12 - SUMMARIZE THE COMPLETE PERFORMANCE INVESTIGATION

```
Summarize the complete SQL performance troubleshooting investigation performed in PROMPTS 1 through 11.

Describe the investigation in chronological troubleshooting order.

Do not hard-code Query IDs or Plan IDs in the general troubleshooting methodology.

Refer to them as the query and plans identified during the investigation, while mentioning the actual identifiers only when
describing the specific evidence from this execution.

Explain:

* Which performance resource was initially identified as the main contributor
* Which query was identified as the main contributor
* Why that query had a high overall performance impact
* How Query Store showed that its behavior changed over time
* What was learned by comparing its execution plans
* Why an execution plan change alone could not explain the beginning of the degradation
* What evidence showed that the same plan was performing dramatically more work per execution
* What was learned from the query predicates and runtime behavior
* What was learned about selectivity and data distribution
* What was learned from statistics
```

```
* What was learned from the existing indexes
* Which corrective action was identified
* Why the proposed index design matches the query access pattern
* How Query Store should be used to validate the change

Separate the response into:

1. Initial symptom
2. Evidence collected
3. Investigation path
4. Diagnosis
5. Recommended corrective action
6. Validation plan

Clearly distinguish confirmed evidence from inference.

Do not claim that a proposed change is successful until before-and-after Query Store evidence demonstrates the improvement.
```

OUTPUT:

1. Initial symptom

Item	Finding
Primary signal from initial Query Store overview	**CPU** was the dominant resource contributor.
Why it mattered	Total CPU concentration was much higher than other metrics, so CPU was selected as the first investigation axis.

2. Evidence collected

Evidence type	Confirmed evidence from this execution
Main contributing query	Query Store identified **`query_id = 1`**, object **`[dbo].[GiveNotes]`**.
Query shape (first 100 chars)	`SELECT count(Name), name ... FROM Notes WHERE ID BETWEEN @N2 AND @N1 GROUP BY Name`
Overall impact	~**1,382,706 ms total CPU**, ~11,517 executions, 4 plans.
Plan-level spread	Plans had extreme cost spread (best to worst avg CPU/exec difference > 2,900x).
Same-plan degradation	For **plan 14**, avg CPU/exec rose **280.39 -> 2,564.88 ms**; avg reads/exec **4,224 -> 34,738**; avg duration/exec **43.95 -> 339.61 ms**.
Time behavior	Degradation continued while the same plan ID remained active (not only at plan transition points).
Wait profile	Parallelism/lock waits increased in degraded intervals, alongside major logical-read growth.
Table/index context	`dbo.Notes` had clustered PK on `id2`; no confirmed index aligned to `ID BETWEEN ...` + `GROUP BY NAME` pattern.

3. Investigation path (chronological)

Step	What was done	Output/learning
1	Built weighted Query Store overview across plans/intervals	Established dominant resource (CPU) and top contributor query.
2	Isolated top query and all its plans	Found material plan-to-plan cost differences.
3	Compared runtime intervals per plan	Confirmed severe within-plan degradation over time (same plan ID).
4	Tested “more waits vs more work” hypothesis	Reads/exec and CPU/exec both surged, supporting “more work per execution.”
5	Ran read-only validation batch (plans, intervals, waits, tables, indexes, stats, histogram, compile params)	Collected supporting metadata and ruled out simplistic “plan change only” explanation.
6	Derived corrective design target	Access path should align with `ID` range predicate and output/grouping requirements.
7	Defined Query Store before/after validation method	Established objective success/failure criteria beyond duration alone.

4. Diagnosis

Confirmed evidence

- The identified query is a high-impact CPU consumer.
- Its cost changed dramatically over time.
- The same plan ID showed large increases in CPU, duration, and logical reads per execution.
- Plan change alone cannot explain the full degradation pattern.

Supported inference

- The strongest explanation is **variable work volume per execution** (different effective range/selectivity under `ID BETWEEN @N2 AND @N1`) with an access path not well aligned to that predicate/grouping pattern.
- Increased waits are likely **secondary amplifiers**, not sole root cause, because logical reads per execution also increased sharply.

Remaining uncertainty

- Runtime parameter values per interval were not captured (compiled values were visible; runtime values were null in plan XML extraction).
- Stats/index evidence for all relevant columns was not fully complete in returned grids, so freshness/selectivity modeling uncertainty remains.

5. Recommended corrective action

Action	Why it matches evidence
Add a nonclustered index with **leading key on `ID`** and include/group support for `NAME`	Aligns with range predicate (`ID BETWEEN ...`) and reduces page reads/work needed for grouping/output.
Evaluate supplemental statistics for `ID` + `NAME` correlation	Helps estimation quality where `NAME` is involved in grouping and `ID` drives range filtering.

Action	Why it matches evidence
Evaluate overlapping auto stats after change (do not auto-drop)	Avoid redundant maintenance while preserving useful independent stats.

Why this design fits the access pattern:
The query filters by `ID` and groups by `NAME`. A path keyed on `ID` enables efficient range access; carrying `NAME` at leaf level reduces extra lookups and should lower logical reads, then CPU and duration.

6. Validation plan (Query Store)

Validation area	Required comparison (before vs after)
Core metrics	Executions, avg/total CPU, avg/total duration, avg/total logical reads
Plan behavior	Plan IDs used, plan mix, whether a new worse plan appears
Time behavior	Interval-by-interval trend, outlier intervals
Variability	Max/min or spread of interval avg CPU, duration, reads

Convincing improvement requires:

- Lower avg logical reads/exec and avg CPU/exec (primary),
- Lower totals at comparable execution volume,
- No new dominant worse-cost plan,
- Equal or better stability across intervals.

Important: no change should be declared successful until Query Store before/after evidence confirms these outcomes.